Faculty of Electrical Engineering and Computer Science

Bachelor's Thesis

A Scan-Testable RISC-V Core Implementation using SystemVerilog

Author	Marko Gjorgjievski	33529
Supervisors	Prof. Dr.-Ing. Andreas Siggelkow	Prof. Dr. rer. nat. Markus Pfeil

03. 04. 2025

Declaration

I hereby declare that I have prepared this thesis independently and without unauthorized assistance. All sources and resources used have been fully and accurately cited, and any content derived from the work of others–whether quoted verbatim or adapted–has been clearly identified as such.

.....

Marko Gjorgjievski

Friedrichshafen, 03. 04. 2025

Abstract

Highlighting the importance of test-injection methods on newly developed RISC-V cores, specifically the IEEE 1149.1 standard developed by the JTAG group and the pre-requisites for making such cores scan-testable i.e. making a test-injection method applicable. This thesis follows the developement of a scan-testable RISC-V core following the base integer set RV32I standard using the RTL hardware description language SystemVerilog. After a completed interconnected top-level is written, it is then compiled, simulated and tested through Vivado's software suite. When a functional simulated, synthesizable and scan-testable core is developed, the Cadence Modus DFT Software solution is used for test-injection. It inserts boundary-scan chains and connects them to JTAG-compliant Test Access Port (TAP) controllers with which the test structures can be accessed with. This particular architecture exemplifies design methods for applying IEEE 1149.1-based tests in RISC-V designs.

Contents

List of Figures and Tables

List of Figures

List of Tables

1 Introduction to the RISC-V ISA

1.1 Defining Instruction Set Architecture

The instruction set architecture (ISA) is the interface between hardware and the lowest-level software. It includes the instructions the hardware can execute, the registers it provides, and the ways it accesses memory [1]. A device that can execute instructions described by that ISA, such as central processing unit (CPU), is called an implementation of that ISA.

1.2 Defining Microarchitecture

There are numerous ways in which an implementation of an ISA can be made, which does not have to rely on the characteristics of any single implementation. It only specifies the behavior of machine code running on those implementations, which in turn provides binary compatibility, meaning the same executable code running on different computer systems.

Characteristics such as physical size, performance and cost may differ between multiple implementations depending on what the designer is optimizing the implementation for. Optimization for those parameters can be done by organizing the constituent parts that are included with the specific ISA (instructions, registers, execution model) in the processor. This organization is called a microarchitecture. It is important to define, as it sets the stage for an analysis of an implemented microarchitecture in a later chapter.

1.3 Classification of ISAs

There are different ways to classify ISAs, one of which is architectural complexity.

Architectural complexity divides ISAs into the following categories:

- **CISC**: Complex Instruction Set Computer
- **RISC**: Reduced Instruction Set Computer
- **VLIW**: Very Long Instruction Word
- **EPIC**: Explicitly Parallel Instruction Computer
- **MISC**: Minimal Instruction Set Computer
- **OISC**: One Instruction Set Computer

The most commonly used ISAs based on this classification are CISC and RISC types.

Complex **I**nstruction **S**et **C**omputer (CISC) is an ISA in which several low-level processes can be executed with a single instruction. The defining difference between CISC and RISC architectures is that most CISC designs don't have a uniform instruction length and their store and load operations are usually not separated from arithmetic instructions, where in RISC they would be. They often contain specific instructions that are somewhat rarely used and only by some programs in which they can optimize certain program flows.

Nowadays CISC architectures like x86 are most prominent and are still widely used in desktop applications, with large chip manufacturers such as Intel and AMD holding around 55% of the global processor market [2].

Reduced **I**nstruction **S**et **C**omputer (RISC) is an ISA in which the individual instructions are written with simplicity in mind. Compared to a CISC architecture, RISC systems usually require more instructions in order to execute a given task, because they are written in a shorter and simpler format. The idea is to optimize for execution speed of those instructions and implementing an instruction pipeline [3] (execution of instructions in parallel) which is simpler to establish given the shorter format.

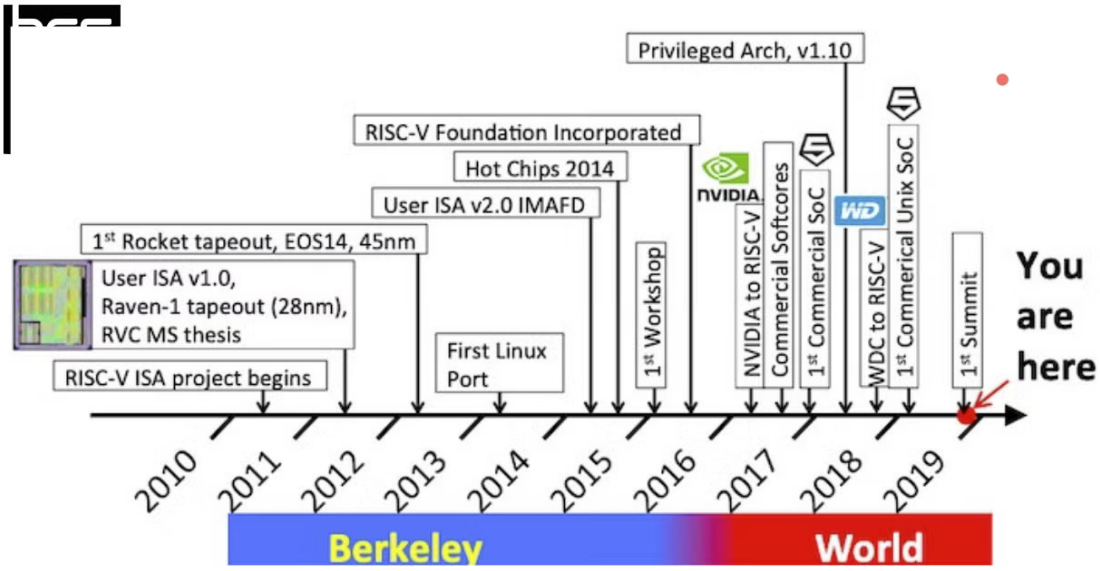
Although RISC architectures are still behind CISC ISAs when it comes to global market share, sitting around 20–23% as of 2025 [2], they are expected to quickly gain traction in the following years, with some sources predicting a growth of over 30% in compound annual growth rate (CAGR) between 2023 and 2030 for the RISC-V market [4]. As more companies are starting to see the value in the RISC-V architecture, some of which can be attributed to its open-source and royalty-free licenses, where most CISC architectures like x86 are proprietary, meaning lack of publicly available documentation, support, needing permission or to pay to use and implement them. Another family of RISC ISAs called ARM are also increasing their market share year-round, with an already dominant 95% share in the smartphone market [5].

1.4 A Brief History of RISC

In the 1970's there was a growing necessity for automating the process of telephone connections, as back then connecting phone lines required human intervention. In 1974, a team of researchers at IBM led by John Cocke started investigating ways to implement a so called telephone exchange controller which could potentially connect 300 calls per second (or a million per hour). In 1980, Cocke's team managed to produce a prototype computer with a RISC architecture, named IBM 801. It's CPU could execute simpler instructions in only one clock cycle. It was also used in other IBM hardware such as IBM ROMP processor in the IBM RISC Technology Personal Computer in 1986 [6].

Later on in the 1980s, a team at UC Berkeley with David Patterson and Stanford with John Hennessy further developed RISC concepts and promoted the architecture, with Patterson's team coining the term "RISC", publishing papers that demonstrated advantages over CISC architectures. In 1984, Hennessy founded MIPS Computer Systems, which was a company specializing in RISC-based processors in workstations and embedded systems. While in 1985, originally a joint venture between Acorn Computer, Apple and VLSI technology, developed the first ARM architecture, aiming for power efficient and simplistic designs. Nowadays ARM chips are virtually everywhere and in almost every handheld device [7].

UC Berkeley expanded on their RISC design with their latest iteration developed in 2010, named RISC-V, aiming for an open-source ISA with an emphasis on flexibility and extensibility with its design [8]. Please refer to the figures 1.1 and 1.2 for the expanded timeline of the RISC-V project as well as Apple's own transition from CISC to RISC CPU chips.



Timeline of RISC-V development given at the RISC-V International's "State of the Union" address in 2019.

Figure 1.1 RISC-V History Timeline [8]

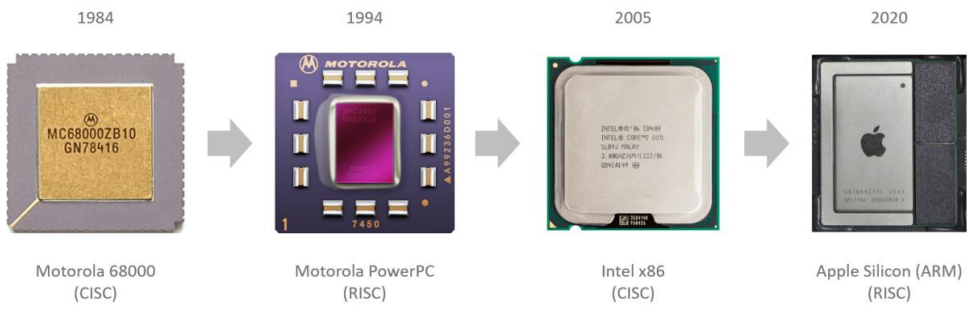


Figure 1.2 Apple Macintosh CPU Transition Timeline [9]

1.5 RISC-V Overview

In the following section, we briefly revisit RISC-V and examine the design philosophy behind the architecture as conceived by Professor Krste Asanović and his graduate students Yunsup Lee and Andrew Waterman.

1.5.1 Goals of RISC-V

RISC-V is a new instruction-set architecture (ISA) that was originally designed to support computer architecture research and education, with the added goal in mind of becoming an open and free architecture for industry implementation [10, p. 10].
Some of the goals in defining RISC-V include:

- A completely open ISA that is freely available to academia and the industry.
- A real ISA suitable for direct native hardware implementation.
- A general purpose ISA that isn't designed for any specific microarchitecture style or implementation technology.
- An ISA separated into a small base integer set, with the option of using standard extensions.
- Support for the revised 2008 IEEE-754 floating-point standard.

There are currently four base ISAs that are a part of RISC-V, which are RV32I, RV64I, RV32E and RV64E [10, p. 13].
Each base integer instruction set is characterized by:

1. Integer registers width
2. Corresponding space of address size
3. Number of integers

There are two primary base integer variants, RV32I and RV64I, which provide 32-bit and 64-bit address spaces respectively. The RV32E and RV64E variants are used to support small microcontrollers and have their number of registers halved. To support general software development, RISC-V's extensibility allows for incorporating different standard extensions depending on the type of functionality that is needed.

Standard extensions are noted by RV32 or RV64 (depending on register width) with the suffix:

- "I" : capable of integer computational instructions, loads, stores and control-flow.
- "M" : capable of multiplication and division of values held in integer registers.
- "A" : capable of atomically reading, modifying and writing memory for inter-processor synchronization.
- "F" : adds floating-point registers, single precision computational instructions, loads and stores.
- "D" : expands floating-point registers, double precision computational instructions, loads and stores.
- "C" : provides narrower 16-bit forms of common instructions

Within the scope of this thesis, the focus is solely on the RV32I base integer set, as it is currently one of the primary variants and most commonly used ISA in the RISC-V family.

1.5.2 RV32I Base Integer Instruction Set

RV32I is a design that is meant to be sufficient for the purpose of forming a compiler target, to support modern operating system environments and to reduce the hardware required in a minimal implementation. It contains 38 base instructions (if the ECALL/EBREAK instructions are covered with a single SYSTEM hardware instruction and FENCE as a NOP).
This base ISA also has 32 x registers that are all 32-bit wide, i.e. $\mathrm{XLEN} = 32$ (the XLEN term referring to the width of an integer register in bits). Register $x0$ is always hardwired with all bits equal to 0, while registers **x1**–**x31** are general purpose registers that hold values which instructions interpret as a collection of boolean values, two's complement signed binary or unsigned binary integers. There is also one additional register: the program counter **pc** which holds the address of the current instruction. Below is a table 1.1 of all available registers in RV32I:

x0/zero	x1/ra	x2/sp	x3/gp	x4/tp	x5/t0	x6/t1	x7/t2
x8/s0/fp	x9/s1	x10/a0	x11/a1	x12/a2	x13/a3	x14/a4	x15/a5
x16/a6	x17/a7	x18/s2	x19/s3	x20/s4	x21/s5	x22/s6	x23/s7
x24/s8	x25/s9	x26/s10	x27/s11	x28/t3	x29/t4	x30/t5	x31/t6
pc							

Table 1.1 RV32I Base Registers

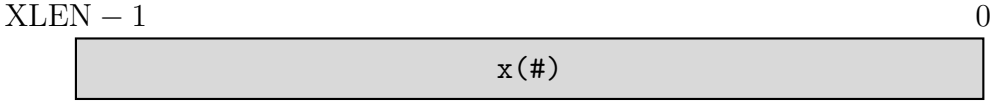


Figure 1.3 Register Bit Width

In base RV32I, there are four core instruction formats available (R/I/S/U). All are fixed 32-bits in length. The base ISA has IALIGN $= 32$, meaning that instructions must be aligned on a four-byte boundary in memory.
The ISA keeps the sources (*rs1* and *rs2*) and the destination (*rd*) registers at the same position in all formats to simplify decoding. Immediates are always sign-extended, and are generally packed towards the leftmost available bits in the instruction and have been allocated to reduce hardware complexity. The sign bit for all immediates is always in bit 31 of the instruction to speed sign-extension circuitry. See the formats below in figure 1.4.

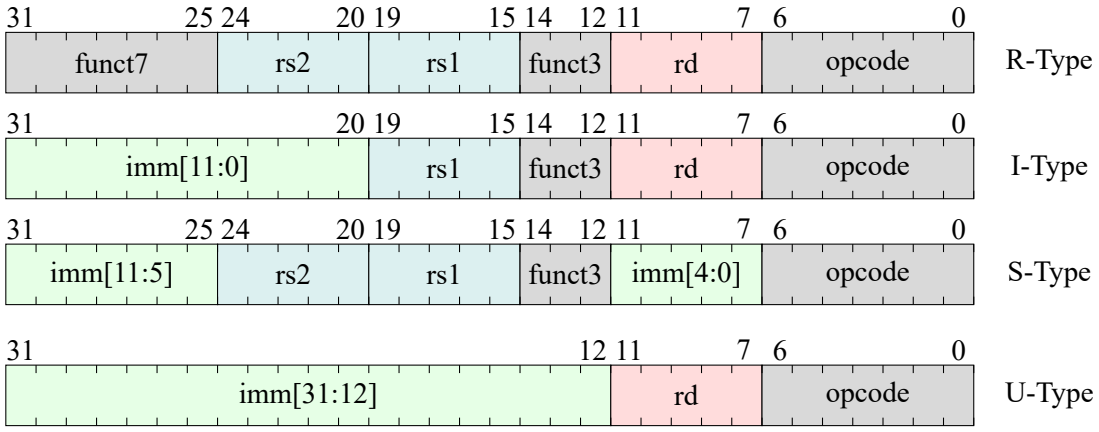


Figure 1.4 Base Instruction Formats

There are a further two variants of instruction formats (B/J) when it comes to handling immediates, shown in the figure 1.5 below.

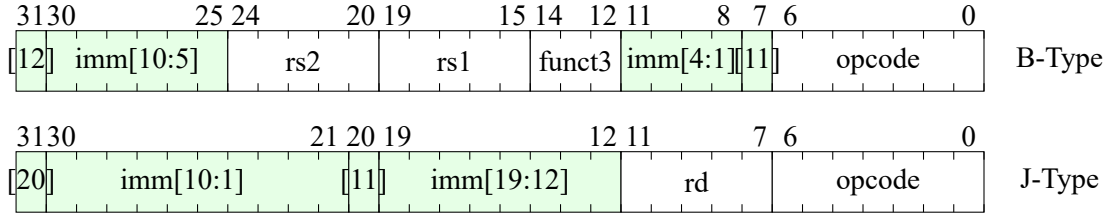


Figure 1.5 Immediate Instruction Formats

Having covered the base and immediate instruction formats of RV32I, the development of a microarchitecture for the core can now proceed.

2 Implementation of a scan-testable RISC-V core

The section follows the implementation of a microarchitecture for the scan-testable RISC-V core [11]. For this particular design, three attributes for simplicity, functionality and scan-testability are sought for.
To satisfy these conditions, a minimalist approach to organizing the microarchitecture with only the essential functional elements would suffice. The functional elements used in this design are:

- Register File - a module describing the design of RV32I registers: 32-bit wide 32 registers.
- Arithmetic Logic Unit (ALU) - this module handles logic, arithmetic and bit-wise operations.
- Instruction Memory - this module contains the read-only memory (ROM) block for storing instructions.
- Data Memory - this module contains the data memory block (RAM) for storing data during program execution.
- Sign Extension Unit - this module derives immediate values from an instruction's fields.
- Branch Unit - this module alters the instruction execution order.
- Instruction Decoder - this module decodes the selected instruction and modifies the appropriate control signals.

A set of tools for describing, simulating, testing and synthesizing this design is also necessary. The tools used in achieving the aforementioned goals are:

- Visual Studio Code - A integrated development environment (IDE) for writing the design in. Supports features such as syntax highlighting for hardware description languages, workflow organization, seamless integration with version control software such as git [12].
- SystemVerilog - also known as IEEE 1800, is a hardware description and verification language that is used to design, simulate, test and implement digital electronic elements and systems. It is a direction extension of Verilog-2005, therefore, Verilog is a subset of SystemVerilog and supports all its features. Both languages are used for designing register-transfer level (RTL) circuits, which is a design abstraction that models synchronous digital circuits in terms of flow of digital signals (data) between hardware registers, and the logical operation performed on those signals. Another important feature of SystemVerilog is its extensive use of object-oriented programming (OOP) techniques for verification, which makes it closely resemble OOP languages such as Java. These constructs, however, are not synthesizable [13].

- Vivado - also known as Vivado Design Suite is a software suite used for compiling, simulating, synthesizing, placing, routing, and programming HDL designs. Vivado's high-level synthesis compiler enables C, C++ and SystemC programs to be targeted into Xilinx devices without the need to manually create RTL. In this thesis, its relevancy is tied with supporting SystemVerilog features for verification, mainly assertions when creating testbenches for the specific modules in the core and the core itself [14].
- Git - is a free and open-source distributed version control software used to track the development for the core in this project. It supports features for recording edits of files over time, separates development into different branches for different features. Project files are usually stored in repositories, which are either locally saved or remotely present on websites like GitHub [15].
- GitHub - is a proprietary developer platform where users can upload projects as remote repositories, where multiple collaborators can work on the same project, review code changes and do software versioning control [16]. In the scope of this thesis, GitHub was used for tracking personal code changes.
- Cadence Modus DFT Software Solution - is a design-for-test (DFT) and an automatic test pattern generation (ATPG) tool that supports the IEEE 1149.1 JTAG standard, where it can insert DFT logic [17].

In the context of scan-testability, the chip must satisfy two conditions for a scan-injection from DFT software. For a design to be scan-testable, it must:

- **Use a single clock signal for its architecture.**
- **Use a single reset signal for its architecture.**

Only if these conditions are met, could a scan-chain injection be permissible.
After covering the tools and conditions necessary for this implementation, an analysis of the implemented scan-testable core will be covered in the following sections.

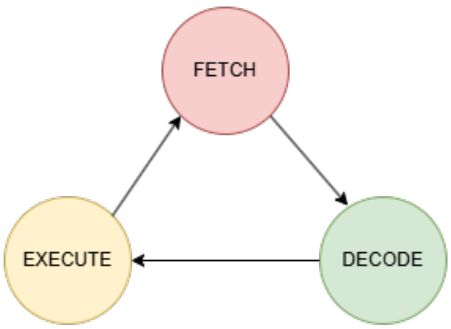


Figure 2.1 An instruction fetch-execute cycle.
The current implementation executes instructions in a single clock cycle.

2.1 Top-Level View

For a functioning core design, all modules need to be interconnected in the top-level with internal signals. Below is detailed functional block diagram 2.2 with all internal elements and wiring present.

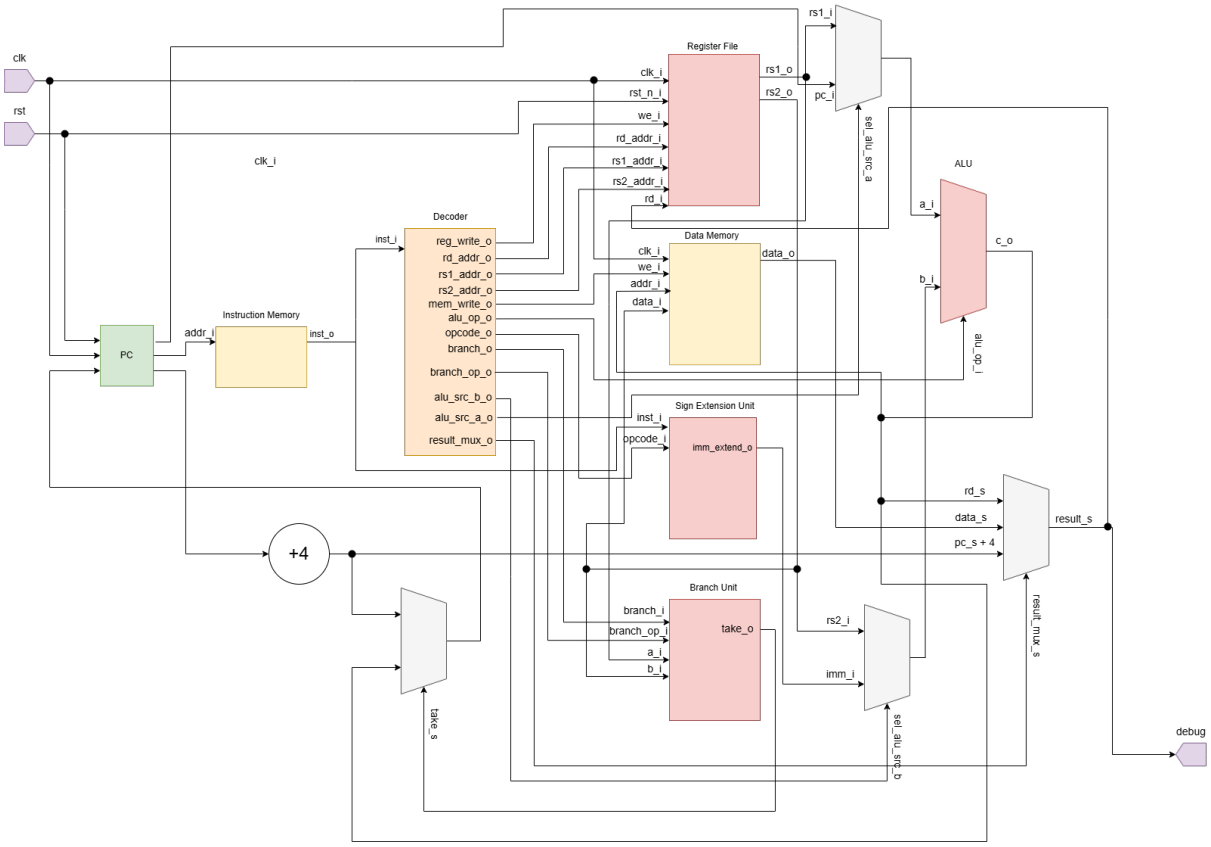


Figure 2.2 Functional Block Diagram of Top-View

Signal	Direction	Description
clk	Input	Clock signal.
rst	Input	Active-low reset.
debug	Output	Instruction execution result.
pc_s [31:0]	Internal	Program Counter.
inst_s [31:0]	Internal	Instruction to decoder.
branch_s	Internal	Branch enable signal.
result_mux_s [1:0]	Internal	Result mux selector.
alu_op_s [5:0]	Internal	ALU operation selector.
mem_write_s	Internal	Data memory write enable.
alu_src_a_s	Internal	ALU source selector.
alu_src_b_s	Internal	ALU source: selector.

reg_write_s	Internal	Register write enable.
take_s	Internal	Branching comparison result.
opcode_s [6:0]	Internal	Instruction operational code.
rs1_addr_s [4:0]	Internal	Register 1 address.
rs2_addr_s [4:0]	Internal	Register 2 address.
rd_addr_s [4:0]	Internal	Register to write address.
sel_alu_src_a_s [31:0]	Internal	ALU source: reg1/pc.
sel_alu_src_b_s [31:0]	Internal	ALU source: reg2/immediate.
immediate_s [31:0]	Internal	Immediate value.
rs1_o [31:0]	Internal	Register 1 output.
rs2_o [31:0]	Internal	Register 2 output.
data_s [31:0]	Internal	Data memory output.
result_s [31:0]	Internal	Debug multiplexer result.
rd_s [31:0]	Internal	ALU result.

Table 2.1 Top-Level Signal List

2.2 Package File

Package file contains all the information necessary for defining register, instruction and opcode data widths, as well as operation codes for the instructions themselves. It is included in every module that uses those parameters. Below is a table describing those values:

Parameter	Value	Description
DATA_WIDTH	32	Register bit-width.
NUM_REGISTER	32	Number of registers.
INST_WIDTH	32	Instruction bit-width.
OP_ALU_NOP	000000	ALU operation code.
OP_ALU_ADD	011001	ALU operation code.
OP_ALU_SUB	011011	ALU operation code.
OP_ALU_AND	011101	ALU operation code.
OP_ALU_OR	011111	ALU operation code.
OP_ALU_XOR	100001	ALU operation code.
OP_ALU_SLT	100011	ALU operation code.
OP_ALU_SLTU	100101	ALU operation code.
OP_ALU_SLL	100111	ALU operation code.
OP_ALU_SRL	101001	ALU operation code.
OP_ALU_SRA	101011	ALU operation code.
BRANCH_BEQ	000	Branching operation code.
BRANCH_BNE	001	Branching operation code.
BRANCH_BLT	100	Branching operation code.
BRANCH_BGE	101	Branching operation code.
BRANCH_BLTU	110	Branching operation code.
BRANCH_BGEU	111	Branching operation code.
BRANCH_JAL_JALR	010	Branching operation code.
OPCODE	7	Instruction operational code bit-width.
OP_LUI	0110111	Instruction operation code.
OP_AUIPC	0010111	Instruction operation code.
OP_JAL	1101111	Instruction operation code.
OP_JALR	1100111	Instruction operation code.
OP_BRANCH	1100011	Instruction operation code.
OP_LOAD	0000011	Instruction operation code.
OP_STORE	01000011	Instruction operation code.
OP_ALU	0110011	Instruction operation code.
OP_ALUI	0010011	Instruction operation code.
OP_FENCE	0001111	Instruction operation code.
OP_SYSTEM	1110011	Instruction operation code.
FUNCT_7	7	func7 field bit-width.

Table 2.2 Package Configuration Parameter List

2.3 Register File

The register file currently stores all 32 registers present in the base RV32I instruction set. All registers in this set are 32-bit wide. Updating the registers inside the register file is always done within a clocked process. It takes a single clock cycle for a single register to latch onto new data based on the given data from the instruction that the module is being fed from. This includes the input ports that receive that data:

- we_i - a write enable control signal from the decoder. When this signal is high, data can be written to the selected register.
- rd_addr_i - the address of the register the data is being written to.
- rd_i - the data that is being written.

On the transmitting end, two registers are always available to read from whenever the data from the former is being requested. The signals that support this are:

- rs1_addr_i - input for the address of the first register to read data from.
- rs2_addr_i - input for the address of the second register to read data from.
- rs1_ - first available register.
- rs2_o - second available register.

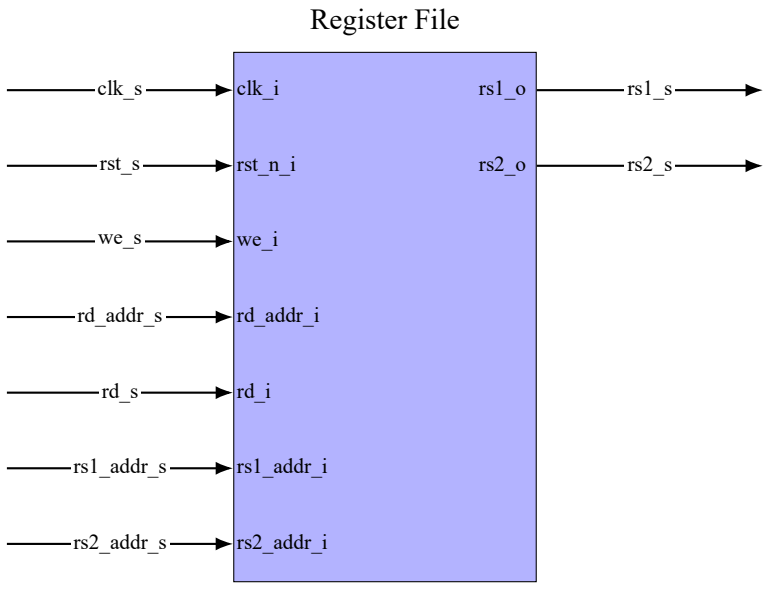


Figure 2.3 Block Diagram of Register File

Signal	Direction	Description
clk_i	Input	Clock signal.
rst_n_i	Input	Active-low reset, affects all registers
we_i	Input	Write enable. When signal is high, writing to registers enabled.
rd_addr_i [4:0]	Input	Address of register to write data.
rd_i [31:0]	Input	Data for writing to register.
rs1_addr_i [4:0]	Input	Address of first register to read from.
rs2_addr_i [4:0]	Input	Address of second register to read from.
rs1_o	Input	First output register.
rs2_i	Input	Second output register.

Table 2.3 Register File Signal List

2.4 Arithmetic Logic Unit

The ALU (Arithmetic logic unit) handles bit-wise, shifting and arithmetic operations for the RV32I core. There are two operands which could cover either register-register operations or register-immediate operations.
The instructions that ALU covers are:

- OP_ALU_ADD - Addition of the operands.
- OP_ALU_SUB - Subtraction of the operands.
- OP_ALU_AND - Bit-wise AND of the operands denoted as “&”.
- OP_ALU_OR - Bit-wise OR of the operands denoted as “|”.
- OP_ALU_XOR - Bit-wise XOR of the operands denoted as “^”.
- OP_ALU_SLT - Signed less than comparison.
- OP_ALU_SLTU - Unsigned less than comparison.
- OP_ALU_SLL - Shift left logical.
- OP_ALU_SRL - Shift right logical.
- OP_ALU_SRA - Shift right arithmetic.

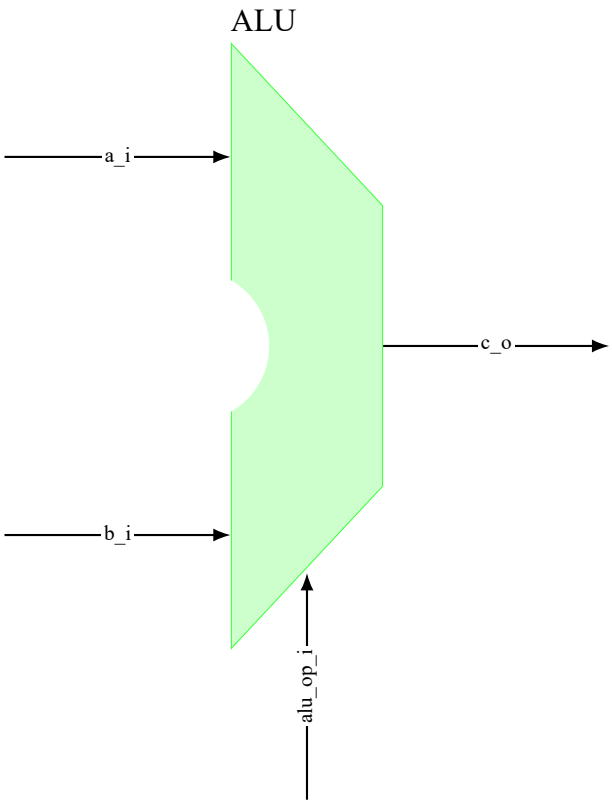


Figure 2.4 Block Diagram of ALU

Signal	Direction	Description
alu_op_i [5:0]	Input	Selector-type signal used for the ALU MUX based on the instruction given.
a_i [31:0]	Input	First operand.
b_i [31:0]	Input	Second operand.
c_o [31:0]	Output	Operation result.

Table 2.4 ALU Signal List

2.5 Sign Extension Unit

The sign extension unit is responsible for deriving immediate values from instruction fields based on the type of instruction the unit is being fed. Please refer to the base instruction formats figures 1.4 and 1.5 to see which fields based on instruction type the sign extension unit derives from, which in this case is the green highlighted imm fields.

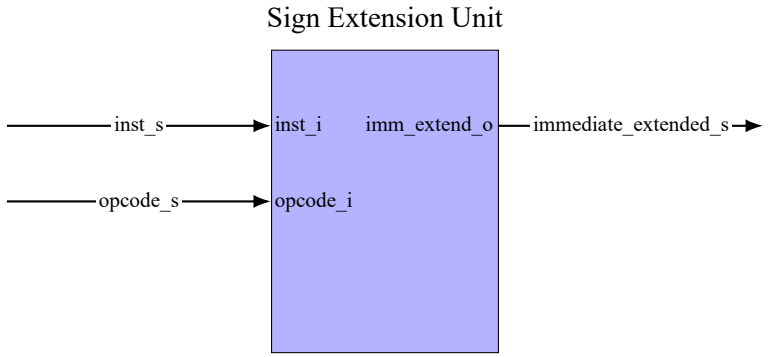


Figure 2.5 Block Diagram of Sign Extension Unit

Signal	Direction	Description
inst_i [31:0]	Input	Instruction (32-bit) that is being used to derive the immediate from.
opcode_i [6:0]	Input	Opcode field which shows the instruction type (R/I/S/U/B/J).
immediate_extended_o [31:0]	Output	Extended 32-bit immediate value.

Table 2.5 Sign Extension Unit Signal List

2.6 Branch Unit

The branch unit is responsible for altering the program execution order depending on the given comparison that is selected through the decoded instruction for the branch. The comparison types covered by the branch unit are:

- BRANCH_BEQ - branch if the two operands are equal.
- BRANCH_BNE - branch if the two operands are not equal.
- BRANCH_BLT - evaluate if the first operand is less than the second.
- BRANCH_BGE - evaluate if the first operand is greater or equal to the second.
- BRANCH_BLTU - compares two unsigned operands for less than.
- BRANCH_BGEU - compares two unsigned operands for greater than or equal.
- BRANCH_JAL_JALR - executes a jump in the instruction flow.

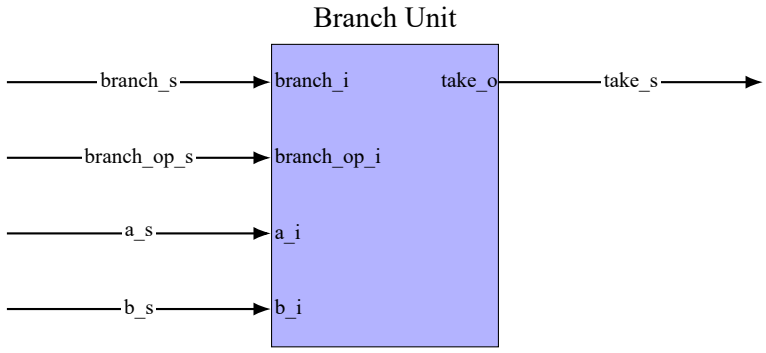


Figure 2.6 Block Diagram of Branch Unit

Signal	Direction	Description
branch_i	Input	Enable signal for branching.
branch_op_i [2:0]	Input	Evaluate which branching operation to execute.
a_i [31:0]	Input	First operand.
b_i [31:0]	Input	Second operand.
take_o	Output	Branching result operation signal, branch if high.

Table 2.6 Branch Unit Signal List

2.7 Memory

Microarchitectures can be classified as different types based on the way they organize memory inside the design. Currently there are two of these types of microarchitectures, the Von Neumann and the Harvard variant. The key distinction between these types is the choice of whether to store the data and the instructions in one memory block, or divide the memory block into two separate blocks, one for data and another for instructions. In the design for this thesis, the choice was made for the Harvard architecture, with a divided memory block.

2.7.1 Instruction Memory

The Instruction Memory is a read-only memory block (ROM) where all 32-bit instructions for the program execution are stored. Currently it can hold 1024x32-bit instructions or in this case $1024 \times 4\text{words} = 4096\text{kiloBytes}$, where $8\text{bits} = 1\text{word}$. Using the PC register we can access the instructions inside this memory, when connected to the address input port and extract the instruction from inst_o. It is also important to note that instruction memory is word-addressable, while addr_i is byte-addressable, meaning it is necessary to right shift the data by 2 to access the correct word for the instruction.

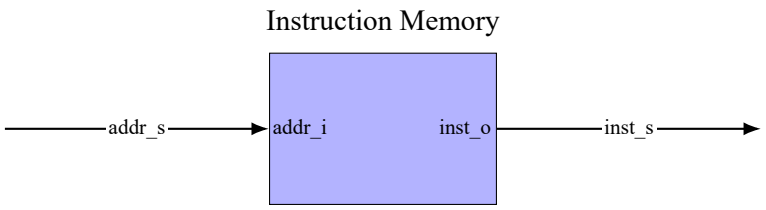


Figure 2.7 Block Diagram of Instruction Memory

Signal	Direction	Description
addr_i [31:0]	Input	Address port for accessing the instructions.
inst_o [31:0]	Output	Resulting instruction from memory.

Table 2.7 Instruction Memory Signal List

2.7.2 Data Memory

Data memory is a memory module that is used for storing and writing data during program execution. It is accessible through load and store instructions. While reading from this memory is concurrent, writing is only accessible through a high write enable signal we_i, and data is written on the positive edge of the clock.

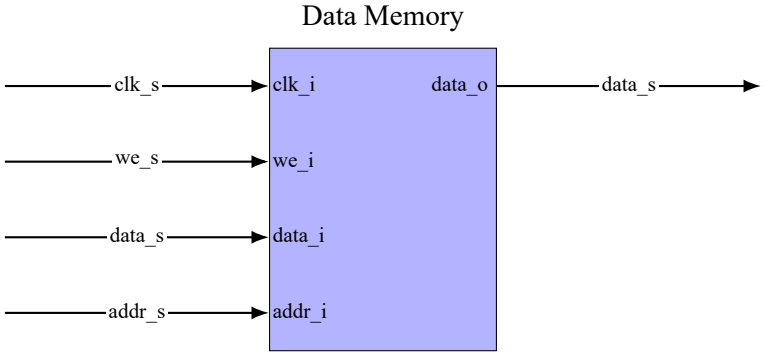


Figure 2.8 Block Diagram of Data Memory

Signal	Direction	Description
clk_i	Input	Clock signal.
we_i	Input	Write enable signal.
data_i [31:0]	Input	Data to be written to the stack.
addr_i [31:0]	Input	Address input for reading/writing the right position in memory.
data_o [31:0]	Output	Output data that can be accessed.

Table 2.8 Data Memory Signal List

2.8 Instruction Decoder

The instruction decoder translates fetched instructions from the instruction memory block into control signals for other functional modules in the chip. These control signals dictate the flow of operation, by enabling or disabling certain functional blocks depending on the decoded instruction. In some designs, this unit is also named as a control unit, because of the properties it possess that were previously mentioned.

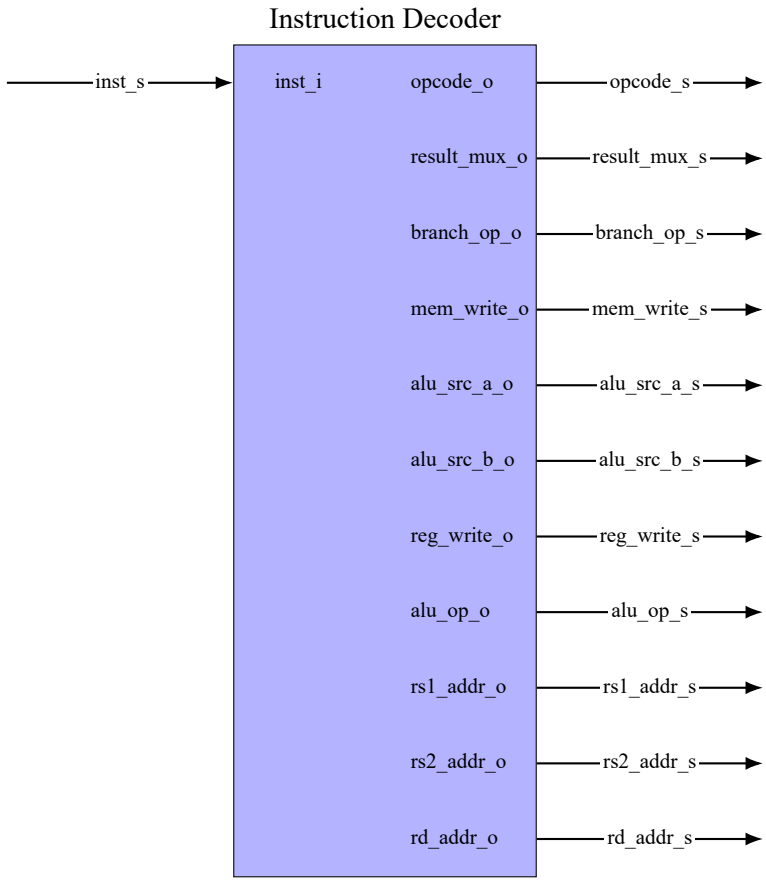


Figure 2.9 Block Diagram of Instruction Decoder

Signal	Direction	Description
inst_i [31:0]	Input	Instruction to be decoded.
opcode_o [6:0]	Output	Resulting opcode.
branch_o	Output	Resulting branch enable.
result_mux_o [1:0]	Output	Selector signal for result mux.
branch_op_o [2:0]	Output	Resulting branch operation.
mem_write_o	Output	Write enable for data memory.
alu_src_a_o	Output	Selector signal for register 1 or PC.
alu_src_b_o	Output	Selector signal for register 2 or immediate.
reg_write_o	Output	Write enable for registers.

alu_op_o [5:0]	Output	Resulting ALU operation.
rs1_addr_o [4:0]	Output	Resulting address for register 1.
rs2_addr_o [4:0]	Output	Resulting address for register 2.
rd_addr_o [4:0]	Output	Resulting data write address for registers.

Table 2.9 Instruction Decoder Signal List

3 Implementation of a TAP Module

With the complete, functioning and scan-testable RV32I core readily available, the Cadence Modus DFT Software Solution can finally be used to inject the scan-chain to test the RISC-V core. In the following section, the concept of Design for Testing will be introduced, an analysis of this test-injection, the standard behind it and architecture itself will be covered.

3.1 Design for Testing

The term Design for Testing refers to the techniques used for testing modern digital circuits for improving the quality and reduce the test cost of former. They are used to simplify test, debug and to diagnose tasks. The most commonly used DFT technique is **scan design**. It implements multiple storage elements into shift registers called **scan chains**, to provide them with external access. Scan design accomplishes this by replacing all selected storage elements with scan cells, with each one having a scan input and scan output port. In the scope of this thesis, a DFT digital design standard that is capable of testing the internal logic of the RV32I core, will be explored [18, p. 69].

3.2 The Digital Boundary Scan Standard IEEE 1149.1

The digital boundary scan standard IEEE 1149.1 defines a boundary-scan architecture and test access protocol for the intent of testing digital integrated circuits and the digital portions of mixed analog/digital integrated circuits. The name boundary-scan comes from the implementation of boundary-scan cells and chaining those cells in a shift register named boundary-scan register. Chips that comply with this standard can be readily integrated into a PCB with their I/O accessible through the boundary scan register. The testing of the internal logic and the interconnects between the chips is enabled through scan-chain that is connected to the I/Os of each chip. It also enables a normal and a test operation mode which in turn enhances the capabilities of design debugging and fault diagnosis.

3.3 Top-Level View - Test Architecture

An example of a scan-injection following the IEEE 1149.1 standard that Cadence Modus DFT Software supports is presented. Boundary-scan cells are attached to the instruction memory module of the developed core, both on the inputs and outputs of the module. In this case, 32 boundary scan cells would be required on both ends or 64 in total to cover every single bit which is accessible on that module. Test data resembling the Program Counter (PC) register can be shifted through the cells through the TDI-TDO line, and then later propagated to the

R2 register during the Update-DR state. The outputs of the input BS-cells would choose an instruction, while the output BS-cells would capture it when running the SAMPLE instruction in conjunction with the Capture-DR state from the TAP-Controller. Please note that in normal scan insertion scenarios memory blocks are usually avoided and the bypass register is used. The reasoning behind this is potential slow down of the testing process. Tools like Cadence Modus do however, support Memory Built-In Self Test (BIST) that enables efficient testing of memory modules [18].

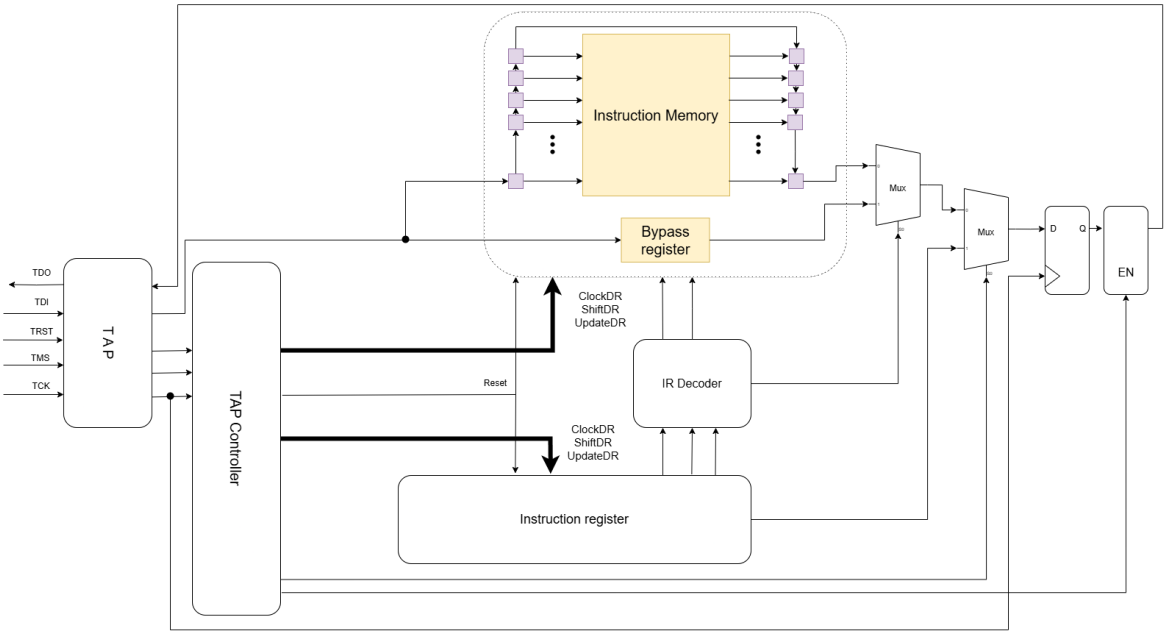


Figure 3.1 Block Diagram of Test Architecture: Testing Instruction Memory

3.4 Data Registers

Depending on the test structure, there are a few data registers implemented in the IEEE 1149.1 standard. The two mandatory data registers are:

- Boundary-Scan Cells
- Bypass register

Furthermore, there are an additional set of optional data registers that can be implemented depending on the needs of the test architect, such as:

- Device-ID register
- Design-specific registers

3.4.1 Boundary-Scan Cells

The Boundary-Scan Cells (BSC) are the fundamental building blocks of the IEEE 1149.1 test architecture. It grants the tester, using the scan-test interface, access to each individual I/O, flip-flops, registers and other elements of the chip. Each scan cell represents a pair of registers connected to each other, while depending on the data direction, are also connected to the I/O pins of internal logic under test respectively. This complete access of every element inside the chip, allows the tester to shift through test data which can be latched by the chip's inputs and then through the same scan-chain can be extracted for further analysis, which is the essence of this test architecture.

Data direction in this case alludes to which register is connected to which pin on the chip. If the boundary scan cell is an input cell, then the register R2 is connected to its respective input pin of the chip, while data is being driven from the register R1 that latches onto data that is either being shifted from TDI through SI, or from external components through the IN port. On the other hand, if the boundary scan cell is an output one, it allows for the register R1 to latch onto the resulting data from the internal circuitry which can be shifted out through the SO-TDO line for further analysis.

The cells also allow for two modes of operation: a normal mode and a test mode. When the normal mode is selected, the internal circuitry operates as normally, no test data is shifted in or out. When the test mode is selected, data from the TDI port can be shifted into the scan-chain for testing, and shifted out through TDO for analysis.

Apart from the mode signal as a given control for the BS-cells, there are three more control signals that control the capturing, shifting and updating operations. These are all functional in the test mode, and the operations for shifting and updating are also available in normal mode. When capturing, the ShiftDR signal is set to 0, a single clock pulse is applied to ClockDR and the data is captured in the R1 register. When shifting, the ShiftDR signal is set to 1, multiple clock pulses are applied to ClockDR, as to shift the data between the R1 registers of the selected boundary-scan cells, ran through the SI-SO line. When updating, the data stored in the R1 register is propagated to the R2 register, then the output of the R2 register is connected to OUT. Below is a block diagram of the BSC design 3.2.

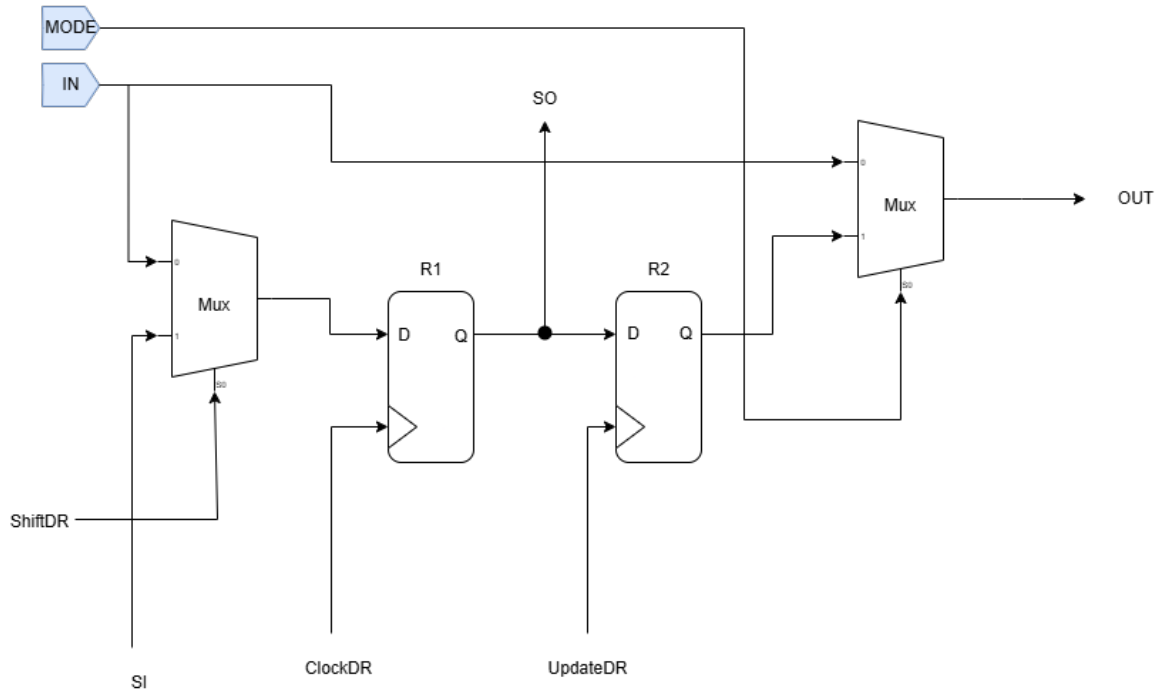


Figure 3.2 Block Diagram of Boundary-Scan Cell [18, p. 566]

3.4.2 Bypass Register

The bypass register is a single-bit register that is used for bypassing the chip when it isn't involved in the current test operation. This can drastically improve test time, by reducing the chain path that is required for shifting in/out test data through TDI-TDO.

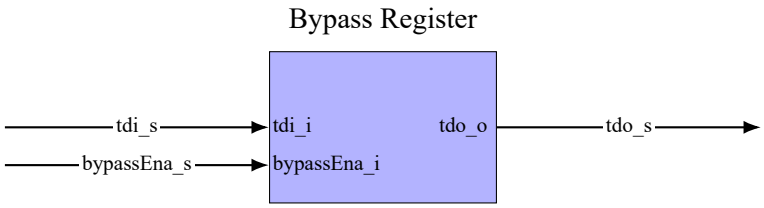


Figure 3.3 Block Diagram of Bypass Register [18, p. 566]

3.5 The TAP-Controller

The TAP-Controller is a 16-state, finite state machine (FSM), that handles nine control signals necessary for the test operations to operate on the appropriate data registers or instruction register. The state transition relies on two signals: TCK and TMS, where TMS is used for determining the next state. The nine control signals are: ClockDR, ShiftDR, UpdateDR, ClockIR, ShiftIR, UpdateIR, Select, TCK, Enable and RST (optional). The main functions of the controller are:

- Resetting the boundary-scan architecture

- Use control signals to load instructions into the instruction register
- Use control signals to provide test functions for capturing and updating of test data.
- Use control signals to shift test data from TDI to TDO.

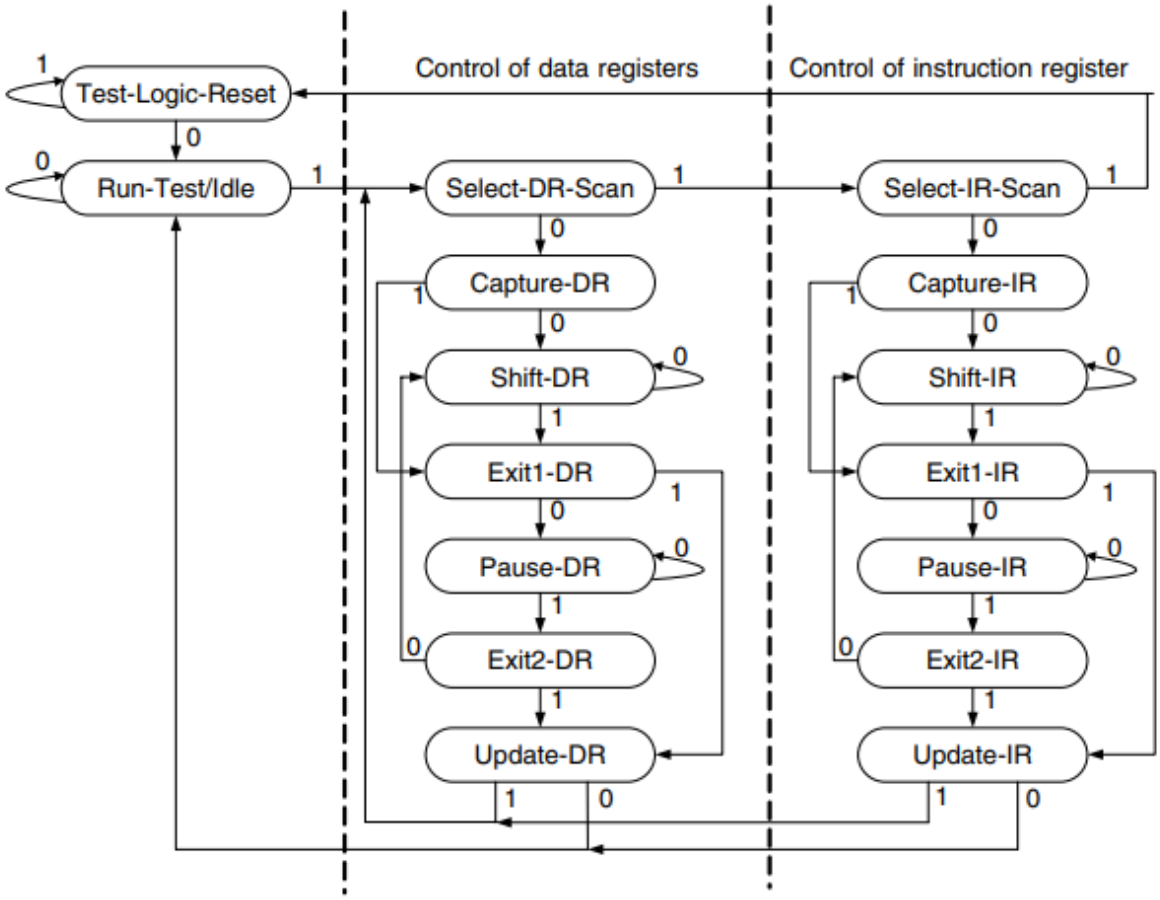


Figure 3.4 TAP Controller State Transition Diagram [18, p. 567]

The state-transition is divided into three section based on registers the states control. The first column covers reset states, the second covers the operations for the data registers and the third one covers the same operations on the instruction register. A brief overview of what happens in each state:

- Test-Logic-Reset - initial starting state, and the state the controller returns to after a reset. Boundary-scan circuitry is disabled, the logic runs in normal mode. The controller can return to this state from any other after 5 consecutive clock cycles where a logic 1 is applied to the TMS signal.
- Run-Test/Idle - test operations are still running in this state and the boundary-scan circuitry could be waiting for those to finish.
- Select-DR-Scan - temporary state, entering data register manipulation.

- Capture-DR - data can be loaded in parallel to data registers, current test results and normal operation status can be captured.
- Shift-DR - test data can be scanned in series through the selected data registers, will remain in this state while TMS $= 0$.
- Exit-DR - temporary state, before entering Pause-DR or Update-DR.
- Pause-DR - boundary-scan logic pauses here to wait for some external operations.
- Exit2-DR - temporary state, either the capturing and shifting has been completed and the Update-DR state can be entered or the Pause-DR state has finished and the controller can return to shifting or capturing new test data.
- Update-DR - allows data propagation from first to second register in boundary scan cell, updated depending on the clocks applied to the second register during this mode. If mode is set to 1 then the output of the second register is connected to the OUT port.

3.6 Instruction Handling

In IEEE 1149.1, instruction handling is done by two registers in which one stores the current instruction and is controlled directly by the TAP controller, while the other decodes the resulting instruction and updates the control signals tied to the data registers for using either the bypass register, sampling or pre-loading test data.

3.6.1 Instruction Register

The Instruction Register resembles the boundary scan cells, in that it is controlled by the TAP controller that instructs capturing, shifting from test data input line and updating the register through it. It stores a 4-bit wide instruction in the ir_s register that is fed to the output whenever the UpdateIR signal is high and always on the negative edge of the test clock.
The four mandatory instructions that the register can hold are:

- **BYPASS** - This instruction handles the bypassEna and mux_dataReg (select bypass register for input) control signals and enables the bypass register for easy shifting of test data from TDI to TDO [18, p. 569].
- **SAMPLE** - This instruction handles the sampleEna and mux_dataReg (select boundary scan cell) signal used for capturing the resulting test data out of the internal logic into the output boundary scan cells which can be fed outside the TDO line [18, p. 569].
- **PRELOAD** - This instruction handles the preloadEna control signal used for shifting the test data through the boundary scan cells from the TDI line through the R1 register from SI and outputting it to the next cell through SO[18, p. 570].

- **EXTEST** - This instruction handles testing of external elements from the core[18, p. 571].

Although the test architecture supports the decoding of **EXTEST**, the control signal is not implemented, again citing the simplicity of the design in which only the instruction memory module is currently being tested with IEEE 1149.1 to showcase how the test injection would apply to the core.
There are also optional instructions used throughout the IEEE 1149.1 standard depending on whether the design being tested has those registers available. Those instructions are:

- **INTEST** - A combination of the instructions SAMPLE and PRELOAD running on a single chip, where test data is shifted to the selected boundary scan cells, register R2 latches onto the shifted data inside the BS-cells in the UpdateDR state, a single clock cycle inside the CaptureDR state is ran to capture the data on the outputs of the chip and then shifted again in the UpdateDR state to analyze the results through TDO [18, p. 572].
- **RUNBIST** - Provides a user-accessible self-test function for all chips that are present on the board. All chips run their self-test concurrently when this instruction is ran, therefore greatly reducing test-time [18, p. 573].
- **CLAMP** - The state of all signals driven by the output pins of a chip will be completely defined by the data stored in the boundary scan register, preferably in a “safe” state. This data is shifted by using the PRELOAD instruction prior to running CLAMP [18, p. 573].
- **IDCODE** - This instruction allows for loading the device-ID register with the data (such as part number, manufacturer’s identity, etc...) that is stored somewhere on the chip itself. Data stored in that register can be shifted out through the TDI-TDO line. If implemented, it should be the default instruction when the chip is reset [18, pp. 573–574].
- **USERCODE** - Allows for loading a 32-bit user-programmable identification code into the device-ID register and then shifted out for analysis. Useful for FPGA chips if programming boundary-scan test logic is not allowed [18, p. 574].
- **HIGHZ** - This instruction puts all output pins of a chip in an inactive-drive state for the purpose of driving the outputs to some desired state to test other chips. It requires the BYPASS register to be the only one active on the TDI-TDO line [574][18].

In the scope of this design, the optional data registers that would support these instructions are not present, nor is it necessary to implement the former, based on the fact that the aim is simplicity and functionality of the test architecture. This also excludes functions that do not use additional registers such as INTEST.

3.6.2 Instruction Decoder

The Instruction Decoder handles a number of control signals that directly enable or disable certain data registers based on the given instruction that is being decoded. This is important for avoiding collision between the outputs of test data out line so that the bypass register and boundary scan cells don't output at the same time, when for example running the PRELOAD instruction.
Below are the listed control signals that the instruction decoder handles:

Signal	Direction	Description
bypassEna	Output	Enables the bypass register.
sampleEna	Output	Enables the register R1 in the boundary scan cell (BSC), sets mode to IN for normal operation of the internal logic and selects BSC for MUX.
preloadEna	Output	Enables the register R1 in the boundary scan cell (BSC), sets mode for SI for test mode that allows shifting for the data from TDI and selects BSC for MUX.
extestEna	Output	Currently not implemented for this test architecture.
mux_dataReg	Output	Selects the appropriate data register based on the given instruction.

Table 3.1 Instruction Decoder Signal List

4 Conclusion

With the complete summarization of the test architecture, a reflection of what has been accomplished within the scope of this thesis can be made, which includes:

- A complete implementation of a RV32I core.
- Compliant with the base ISA outlined in the RISC-V Specifications.
- Single cycle instruction execution.
- Scan-testable pre-requisites met, scan-chain can be injected through DFT software.
- Core can be simulated in RTL simulation software such as Vivado.
- An example implementation of IEEE 1149.1 on the instruction module.

Implementing a processor core is no easy feat. There are many challenges that I faced going into this thesis, spanning from research up to programming individual modules, constructing testbenches and simulating the design. One of the biggest personal challenges was defining the scope of the microarchitecture that would be RV32I compliant. I overcame this with proper research, and finding out what constitutes as a functioning CPU core and which elements it consists of. Other things that were new to me that I had to tackle were: a novel HDL (SystemVerilog), proper version control, properly analyzing ISA specifications, using new simulation software (Vivado), etc...

There is an active GitHub repository of this project available at https://github.com/MarkGJ1/rv32i-bachelor for the readers interested in the code written for this implementation. This is not the end of this project. There are still things to explore both on core and test architecture sides, things like the M extension, the base RV64I implementation, flashing the design onto hardware just to name a few.

I would like to thank my Supervisors Professor Andreas Siggelkow and Professor Markus Pfeil for their unwavering support throughout this thesis as well as my close family.

Bibliography

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. San Francisco, CA, USA: Morgan Kaufmann, 2020.

[2] B. Tran. "The rise of risc-v: Is it a threat to arm and x86? (market growth stats)." Accessed: 26.04.2025. [Online]. Available: https://patentpc.com/blog/the-rise-of-risc-v-is-it-a-threat-to-arm-and-x86-market-growth-stats.

[3] G. N. Crystal Chen and K. Shimano. "Pipelining." Accessed: 26.04.2025. [Online]. Available: https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/about/index.html.

[4] D. Robinson. "Arm reckons it'll own 50% of the datacenter by year's end." Accessed: 26.04.2025. [Online]. Available: https://www.theregister.com/2025/04/01/arm_datacenter_cpu_market/.

[5] K. Köck. "Risc-v vs. arm: Who wins in 8 categories?" Accessed: 26.04.2025. [Online]. Available: https://blog.paessler.com/risc-v-vs-arm-who-wins.

[6] IBM. "Reduced instruction set computer (risc) architecture." Accessed: 26.04.2025. [Online]. Available: https://www.ibm.com/history/risc.

[7] K. Finley. "'computing's nobel prize' winners paved the way to smartphone chips." Accessed: 26.04.2025. [Online]. Available: https://www.wired.com/story/computings-nobel-prize-winners-paved-the-way-to-smartphone-chips/.

[8] R. Foundation. "Where to start with risc-v." Accessed: 26.04.2025. [Online]. Available: https://riscv.org/blog/2021/01/where-to-start-with-risc-v/.

[9] D. M. Jamieson. "The evolution of risc architecture." Accessed: 27.04.2025. [Online]. Available: https://www.epcc.ed.ac.uk/whats-happening/articles/evolution-risc-architecture.

[10] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.2*. Berkeley, CA, USA: RISC-V Foundation, 2017.

[11] Bitspinner. "Rv32i introduction." Accessed: May 13, 2025. [Online]. Available: https://www.bit-spinner.com/rv32i/rv32i-introduction.

[12] Microsoft. "Visual studio code." Accessed: May 13, 2025. [Online]. Available: https://code.visualstudio.com/.

[13] IEEE Computer Society, *Ieee standard for systemverilog–unified hardware design, specification, and verification language*, https://ieeexplore.ieee.org/document/8299595, IEEE Std 1800-2017, Accessed: May 13, 2025, 2017.

[14] I. Xilinx. "Vivado design suite." Accessed: May 13, 2025. [Online]. Available: https://www.xilinx.com/products/design-tools/vivado.html.

[15] L. Torvalds. "Git - distributed version control system." Accessed: May 13, 2025. [Online]. Available: https://git-scm.com/.

[16] I. GitHub. "Github - where the world builds software." Accessed: May 13, 2025. [Online]. Available: https://github.com/.

[17] I. Cadence Design Systems. "Cadence modus dft software solution." Accessed: 10.05.2025. [Online]. Available: https://www.cadence.com/en_US/home/resources/datasheets/cadence-modus-dft-software-solution-ds.html.

[18] L.-T. Wang, C.-W. Wu, and X. Wen, *VLSI Test Principles and Architectures: Design for Testability* (Systems on Silicon). San Francisco, CA, USA: Morgan Kaufmann, 2006, ISBN: 9780123705976.

Appendix

List of Appendices

A List of Abbreviations

- CLK - Clock signal
- RST - Reset signal
- BSC - Boundary Scan Cell
- IEEE - Institute of Electrical and Electronics Engineers
- CISC - Complex Instruction Set Computer
- RISC - Reduced Instruction Set Computer
- VLIW - Very Long Instruction Word
- EPIC - Explicitly Parallel Instruction Computer
- MISC - Minimal Instruction Set Computer
- OISC - One Instruction Set Computer
- ARM - Advanced RISC Machines
- CAGR - Compound Annual Growth Rate
- MIPS - Million instructions per second
- ROMP - Research Office Products Division Micro Processor
- JTAG - Joint Test Action Group
- RTL - Register-Transfer Level
- DFT - Design for Testing
- TAP - Test Access Port
- ISA - Instruction Set Architecture
- CPU - Central Processing Unit
- ALU - Arithmetic Logic Unit
- ROM - Read-only Memory
- RAM - Random Access Memory

B RV32I Instruction List

Currently available instructions in the base RV32I instruction set, which have been implemented in this core.

Mnemonic	Format	Opcode	Description	Syntax
ADD	R	0110011	Add	add rd, rs1, rs2
SUB	R	0110011	Subtract	sub rd, rs1, rs2
SLL	R	0110011	Shift left logical	sll rd, rs1, rs2
SLT	R	0110011	Set less than	slt rd, rs1, rs2
SLTU	R	0110011	Set less than unsigned	sltu rd, rs1, rs2
XOR	R	0110011	Bitwise XOR	xor rd, rs1, rs2
SRL	R	0110011	Shift right logical	srl rd, rs1, rs2
SRA	R	0110011	Shift right arithmetic	sra rd, rs1, rs2
OR	R	0110011	Bitwise OR	or rd, rs1, rs2
AND	R	0110011	Bitwise AND	and rd, rs1, rs2
ADDI	I	0010011	Add immediate	addi rd, rs1, imm
SLTI	I	0010011	Set less than imm	slti rd, rs1, imm
SLTIU	I	0010011	Set less than imm unsigned	sltiu rd, rs1, imm
XORI	I	0010011	XOR immediate	xori rd, rs1, imm
ORI	I	0010011	OR immediate	ori rd, rs1, imm
ANDI	I	0010011	AND immediate	andi rd, rs1, imm
SLLI	I	0010011	Shift left imm	slli rd, rs1, shamt
SRLI	I	0010011	Shift right logical imm	srli rd, rs1, shamt
SRAI	I	0010011	Shift right arith imm	srai rd, rs1, shamt
LB	I	0000011	Load byte	lb rd, offset(rs1)
LH	I	0000011	Load halfword	lh rd, offset(rs1)
LW	I	0000011	Load word	lw rd, offset(rs1)
LBU	I	0000011	Load byte unsigned	lbu rd, offset(rs1)
LHU	I	0000011	Load halfword unsigned	lhu rd, offset(rs1)
SB	S	0100011	Store byte	sb rs2, offset(rs1)
SH	S	0100011	Store halfword	sh rs2, offset(rs1)
SW	S	0100011	Store word	sw rs2, offset(rs1)
BEQ	B	1100011	Branch if equal	beq rs1, rs2, label
BNE	B	1100011	Branch if not equal	bne rs1, rs2, label
BLT	B	1100011	Branch if less than	blt rs1, rs2, label
BGE	B	1100011	Branch if greater or equal	bge rs1, rs2, label
BLTU	B	1100011	Branch if less than unsigned	bltu rs1, rs2, label

BGEU	B	1100011	Branch if greater or equal unsigned	bgeu rs1, rs2, label
LUI	U	0110111	Load upper immediate	lui rd, imm
AUIPC	U	0010111	Add upper imm to PC	auipc rd, imm
JAL	J	1101111	Jump and link	jal rd, label
JALR	I	1100111	Jump and link register	jalr rd, offset(rs1)
ECALL	I	1110011	Environment call	ecall
EBREAK	I	1110011	Environment break	ebreak

Table B.1 RV32I Instruction Set Overview